Escuela Técnica Superior de Ingenieros Industriales (UPM)

Wireless Sensor Networks

Lab 1 Report



Author: Jaime Pérez Pérez - 20259

Date: October 17, 2026

CONTENTS

1 Exercise 1 **3**

2 Exercise 2 **7**

2.1 Senders 7

2.2 Receiver 8

2.3 Network's deployment 8

APPENDICES **12**

A Code used in the first exercise 12

B Code used in Exercise 2 13

B.1 Sender 13

B.2 Receiver 15

1 Exercise 1

This first exercise consists of two different nodes sending messages (packets) to one another. The goal is to toggle the status of a LED (ON/OFF) when each node receives a message.

To achieve it, the first step is to take a look at the code. This proposed solution uses one of the devices' example codes: *"example_broadcast"*, available at:
"home/eclipse-workspace/contiki/examples/rime/example_broadcast.c"

The original example handles all packet transmissions and receptions using broadcast communications. To achieve the exercise's objective, the developed code runs after every packet reception. A simple *if-else* structure checks the current status of the led, switching it and saving the new state.

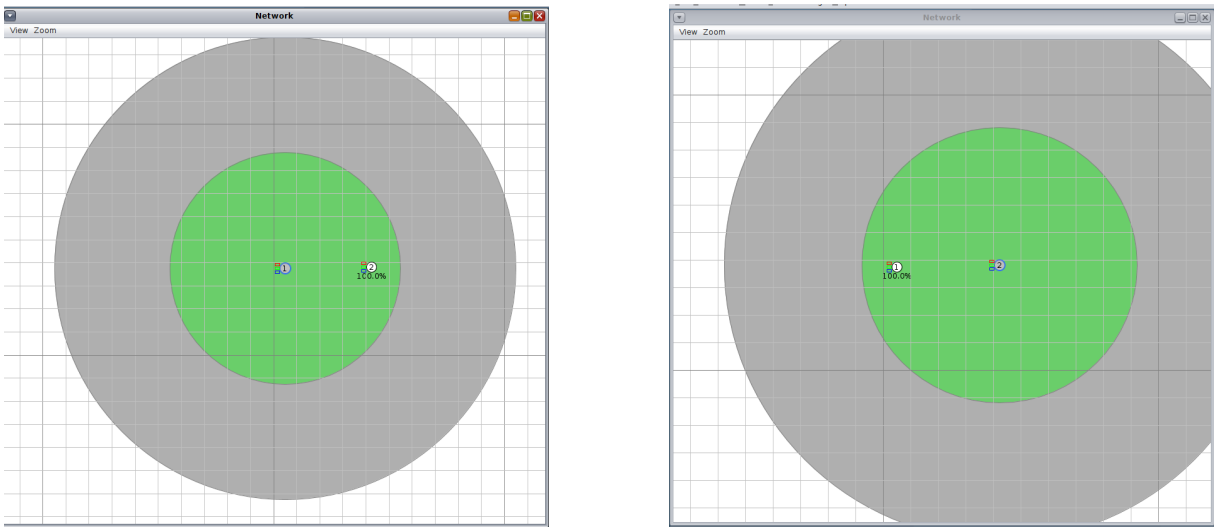
```
1  static void broascast_recv(...){
2
3      //This Function runs on every message reception.
4
5      if(blue_led_on){
6          leds_off(LEDS_BLUE);
7          blue_led_on = 0;
8      }else{
9          leds_on(LEDS_BLUE);
10         blue_led_on = 1;
11     }
12 }
13
14
```

Code 1.1: Developed code. *If-Else* Structure to handle the blue LED status.

(See Appendix A for the full code of this exercise)

Once the code development is finished, this exercise continues by working on the simulation. Two nodes running the previous code are placed on the simulation board, making sure that they both are in range with the other.

This is seen in Figure 1.1. Both nodes are placed inside the other's "green zone", thus guaranteeing that every packet will find a destination.



(a) Node 1 (b) Node 2

Figure 1.1: Node's visibility inside the network

The simulation is then run, and the results are as expected: both nodes send packets between them, and on each reception, they both toggle their blue led status. This is shown step by step in the following images.

1. Both nodes start with their LEDs OFF.

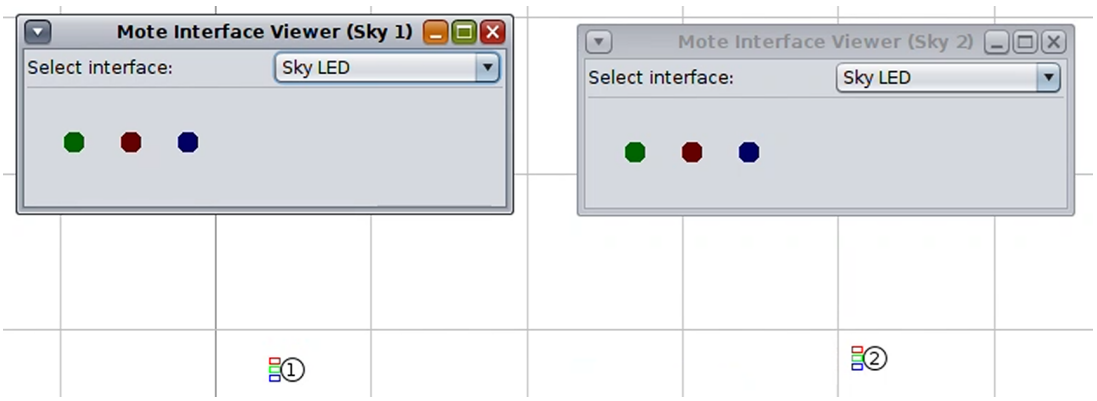


Figure 1.2: Starting state: Both nodes OFF

2. The first node starts the transmissions. In this example, it was node 2 who sent the message first.

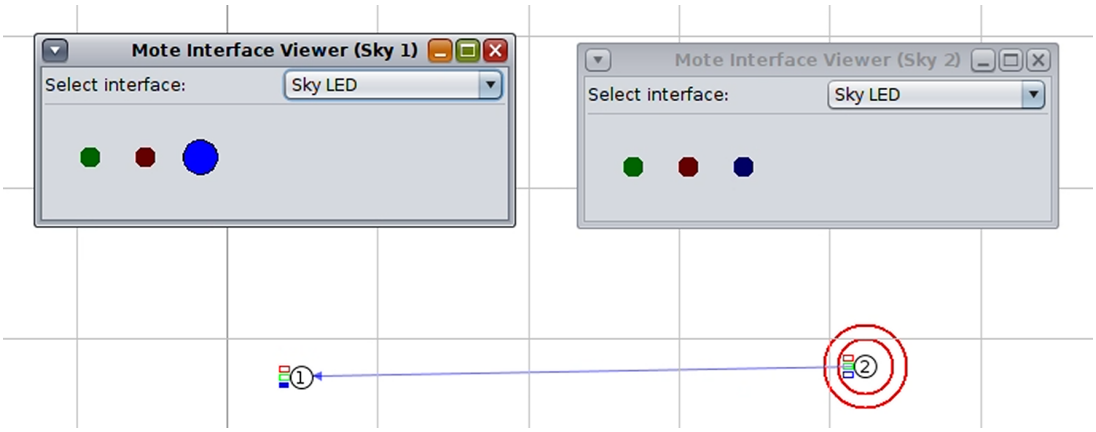


Figure 1.3: Node 1 ON

Upon reception, as seen both in the LED squares placed next to the node and in the LED interface placed on top of the image, node 1 turns on its blue LED.

3. This same sequence is repeated in the opposite direction.

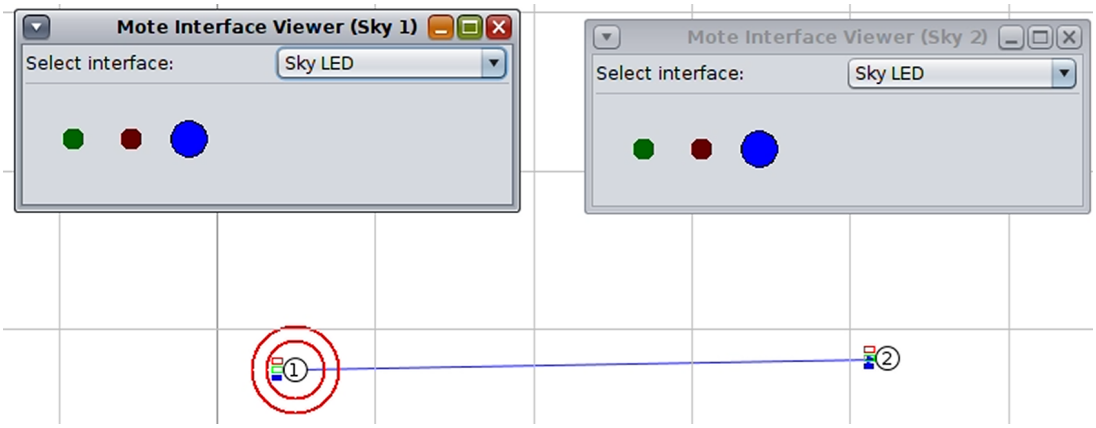


Figure 1.4: Node 2 ON

4. As the simulation continues, when receiving the following packages, both nodes turn their LEDs OFF, and the process restarts from step one.

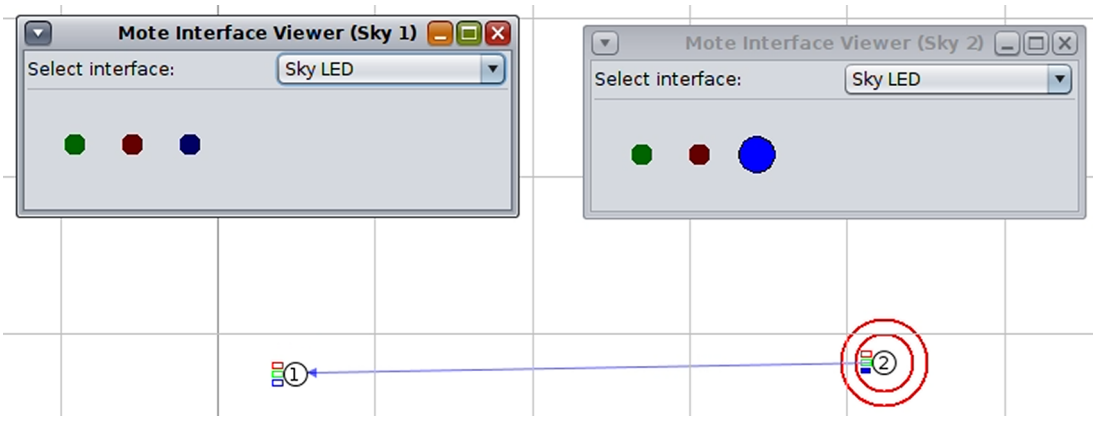


Figure 1.5: Node 1 Off

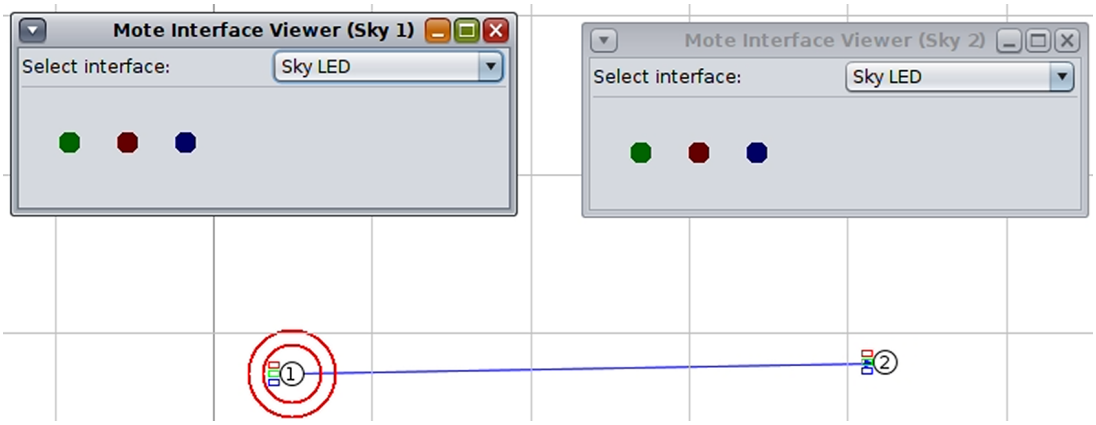


Figure 1.6: Node 2 Off

2 Exercise 2

The second exercise of this lab uses other of the devices' example codes. This time, the nodes will run a routing protocol in charge of finding a path for any communication bewtween them. This example can be found at:
"*home/eclipse-workspace/contiki/examples/IPv6/simple-UDP-RPL/Unicast-sender.c & Unicast-receiver.c*".

The network will be composed of 7 nodes, one of them being a receiver, and the rest will act as senders. All senders will transmit their messages, and the routing protocol will find a way to get them all the way to the receiver (the system's gateway).

One of the goals of this exercise is to demonstrate the functioning of said protocol (UDP). The nodes will be moved around the network, and the results will demonstrate how the senders will always find a path to the receiver.

The content of the messages sent by the nodes is a random simulated temperature within coherent values of ambient temperature. The receiver will be in charge of displaying said temperature to validate the network's functionality.

(See the full sender and receiver codes in Appendix B)

2.1 Senders

All senders will execute the same code. To differenciate them from the receiver when reading the serial monitor, they will all be named 'AAAA'.

Their main task is to generate the random temperature (using the "*random_rand()*" function) and sending it to the receiver using the UDP routing protocol functions. Inside the message, senders will also include their ID, so that the receiver can easily show the origin of each transmission.

In further, more complex exercices, the callback executed whenever a node receives a message also grants access to the transmission's origin. However, in this example, it is easier to include the ID inside the message's payload than printing the whole address to differenciate each sender node.

```
1     (...)
2
3 char buf[20];
4 int temp = 17 + (random_rand()%10);
5 //17 + a random value from 0 to 9.999 .
6
7 sprintf(buf, "T: %d C From: %d", temp,node_id);
8 //Saves the message to be sent, including the temperature and the sender's ID.
9
```

Code 2.1: Message generation: random temperature + sender's ID

2.2 Receiver

Using a callback system, the receiver will print the incoming messages. Whenever a new transmission is received, the callback in charge will run, having access to the sender's address, the port in which the message was received, and more importantly, to the content of the incoming message.

During the simulation, as seen in the following figures, these messages come from all of the devices in the network, thus demonstrating the UDP routing protocol works as it should.

The receiver will extract the payload previously generated by each sender, showing the received temperatures and their origin in real time.

2.3 Network's deployment

The system is simulated using multiple topologies of the network. Several of them will be shown in figures 2.1, 2.2, 2.3 and 2.4.

Said figures will show the layout of the nodes (the topology) used in each scenario. In all of them, the messages flow all the way to the receiver (node 7 in all cases).

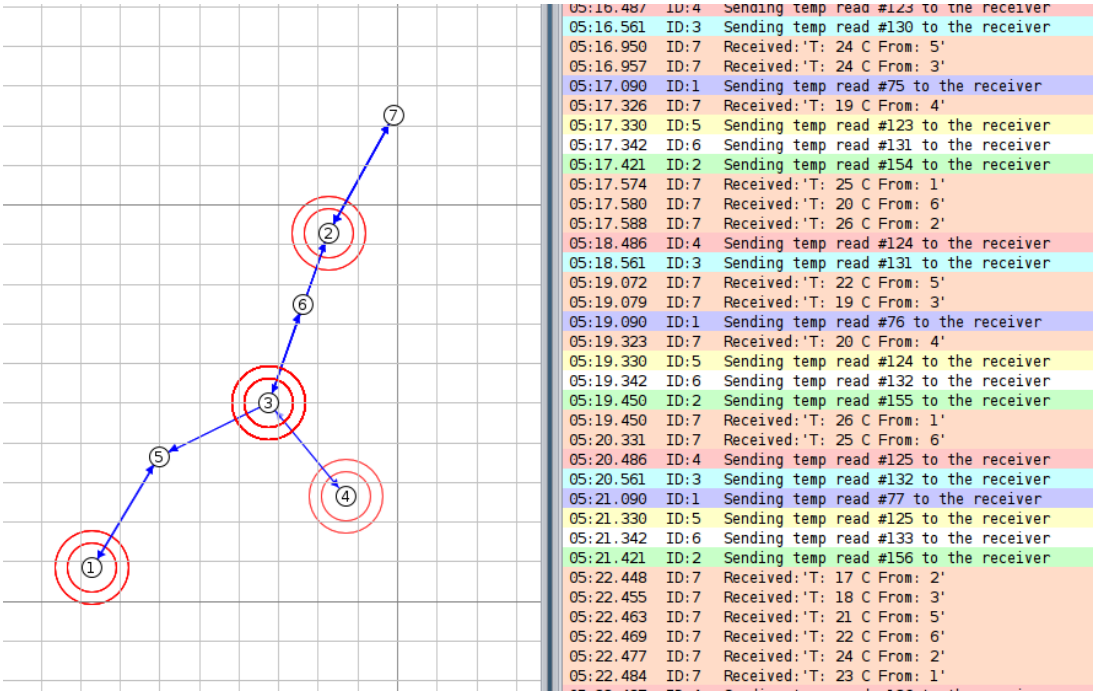


Figure 2.1: Topology 1

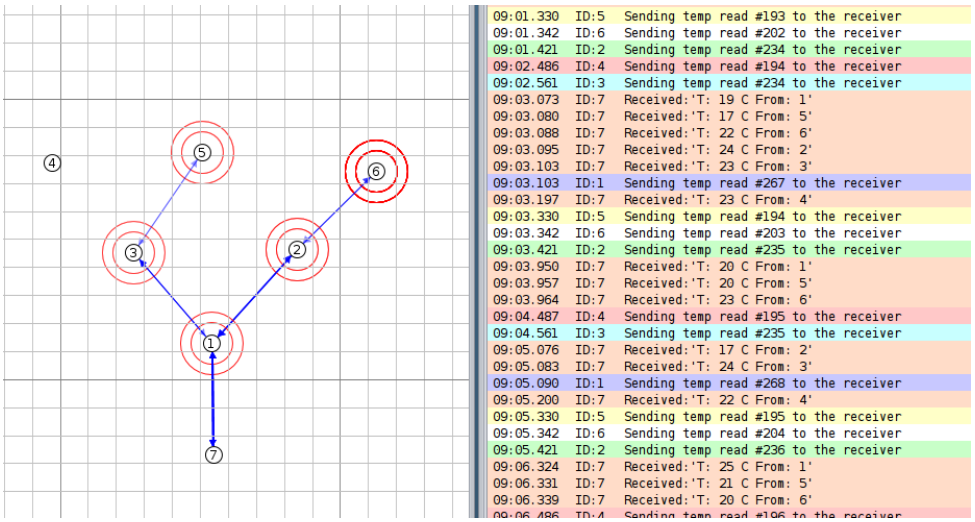


Figure 2.2: Topology 2

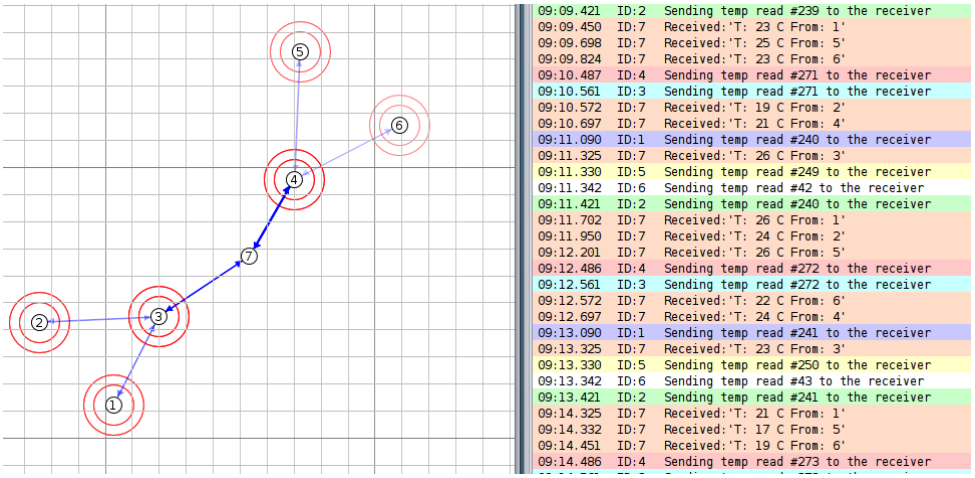


Figure 2.3: Topology 3

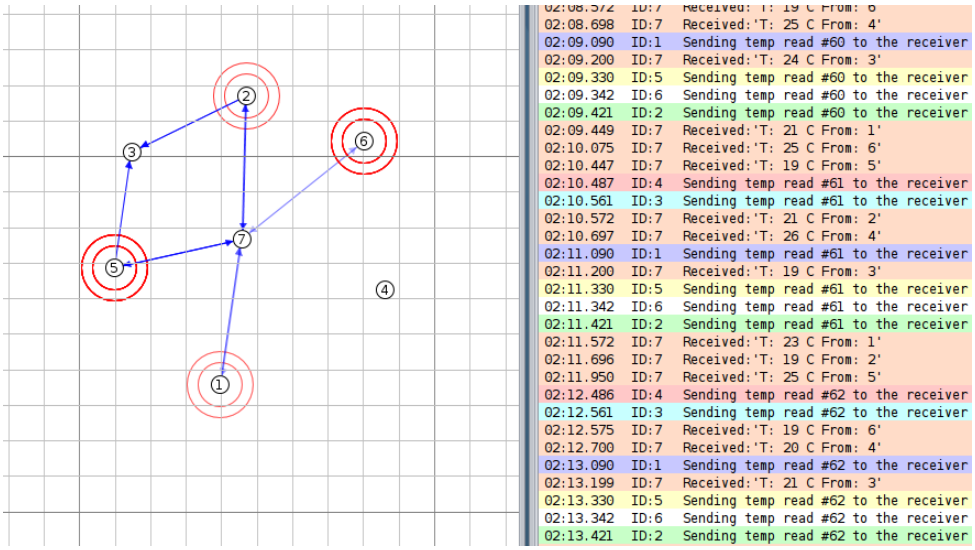


Figure 2.4: Topology 4

See how, in the previous figures, the left part shows (at exact moment the screenshot was taken) how the messages flow within the network, reaching the receiver node in all cases. Following instants would display other nodes sending their messages.

In the right part of all these images, the logs verify that the receiver gets messages from all of the network's nodes. These prints show the random temperature values generate by the senders, as well as their ID.

These few topologies validate the routing protocol used. During this lab's execution, multiple other layouts were tried, obtaining a successfull communication between each sender and the receiver in every case.

APPENDICES

A Code used in the first exercise

```
#include "contiki.h"
#include "net/rime.h"
#include "random.h"
#include "dev/button-sensor.h"
#include "dev/leds.h"
#include <stdio.h>
#define DEBUG 1
uint8_t blue_led_on;

PROCESS(example_broadcast_process, "Broadcast example");
AUTOSTART_PROCESSES(&example_broadcast_process);

static void broadcast_recv(struct broadcast_conn *c, const rimeaddr_t *from){

    //Callback executed on every message reception
    if(DEBUG){
     printf("broadcast message received: %d.%d says: '%s'\n", from->u8[0],
          ↪ from->u8[1], (char *)packetbuf_dataptr());
    }

    //Logic to toggle the blue light:
    if(blue_led_on){
       leds_off(LEDS_BLUE);
       blue_led_on = 0;
    }else{
       leds_on(LEDS_BLUE);
       blue_led_on = 1;
    }
}

static const struct broadcast_callbacks broadcast_call = {broadcast_recv};
static struct broadcast_conn broadcast;

PROCESS_THREAD(example_broadcast_process, ev, data){

    static struct etimer et;
    PROCESS_EXITHANDLER(broadcast_close(&broadcast);)
    PROCESS_BEGIN();
    broadcast_open(&broadcast, 129, &broadcast_call);

    while(1) {

        /* Delay 2-4 seconds */
        etimer_set(&et, CLOCK_SECOND * 4 + random_rand() % (CLOCK_SECOND * 4));

        PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&et));
        packetbuf_copyfrom("Hi!!", 6);

        broadcast_send(&broadcast);
        if(DEBUG) printf("broadcast message sent\n");
    }

    PROCESS_END();
}

```

Code A.2: Exercise 1 full code.

B Code used in Exercise 2

B.1 Sender

```
#include "contiki.h"
#include "lib/random.h"
#include "sys/ctimer.h"
#include "sys/etimer.h"
#include "net/uip.h"
#include "net/uip-ds6.h"
#include "net/uip-debug.h"
#include "sys/node-id.h"
#include "simple-udp.h"
#include "servreg-hack.h"
#include <stdio.h>
#include <string.h>

#define DEBUG 1

#define UDP_PORT 1234
#define SERVICE_ID 190

#define SEND_TIME (2 * CLOCK_SECOND)

static struct simple_udp_connection unicast_connection;

PROCESS(unicast_sender_process, "Unicast sender example process");
AUTOSTART_PROCESSES(&unicast_sender_process);

static void receiver(
            struct simple_udp_connection *c,
                const uip_ipaddr_t *sender_addr,
                uint16_t sender_port,
                const uip_ipaddr_t *receiver_addr,
                uint16_t receiver_port,
                const uint8_t *data,
                uint16_t datalen)
    {

  if(DEBUG) printf("Data received on port %d from port %d with length %d\n",
         receiver_port, sender_port, datalen);
}

static void set_global_address(void){
    uip_ipaddr_t ipaddr;
    int i;
    uint8_t state;

    uip_ip6addr(&ipaddr, 0xAAAA, 0, 0, 0, 0, 0, 0, 0);
    //IPv6 addresses have 8 blocks: 1st has value, the   rest are full of zeroes
    uip_ds6_set_addr_iid(&ipaddr, &uip_lladdr);
    //Changes the last 4 blocks with the device's MAC address.
    uip_ds6_addr_add(&ipaddr, 0, ADDR_AUTOCONF);
    //Writes the built-up address on the node.
}
```

```
51 PROCESS_THREAD(unicast_sender_process, ev, data){
52
53     static struct etimer periodic_timer;
54     uip_ipaddr_t *addr;
55
56     PROCESS_BEGIN();
57     servreg_hack_init();
58     set_global_address();
59
60     simple_udp_register(&unicast_connection, UDP_PORT, NULL, UDP_PORT,
           ↪ receiver);
61
62     //Initial OFFSET to avoid colissions & interferences:
63     etimer_set(&periodic_timer, random_rand() % SEND_TIME);
64     PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&periodic_timer));
65
66     //After the initial random delay, the timer is set to wait exactly 2
           ↪ seconds from now on.
67     etimer_set(&periodic_timer, SEND_TIME);
68
69   while(1) {
70
71     PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&periodic_timer)); //Once the
           ↪ periodic timer finishes
72     etimer_reset(&periodic_timer); //Resets the timer
73
74     addr = servreg_hack_lookup(SERVICE_ID);
75     // (Only the receiver will register this service ID)
76     // All senders, in this reduced example, know the service ID beforehand.
77
78     if(addr != NULL) {
79       static unsigned int message_number;
80       char buf[20];
81
82       int temp =  17 + (random_rand()%10);
83       //17 + a number from 0 to 9.999 (17-27 ºC)
84
85       sprintf(buf, "T: %d C From: %d", temp,node_id);
86
87       if(DEBUG){
88
89         printf("\nSending temp read #%d",message_number);
90         printf(" to the receiver");
91       }
92       message_number++;
93
94       simple_udp_sendto(&unicast_connection, buf, strlen(buf) + 1, addr);
95     } else {
96       printf("No receivers with service id: %d found\n", SERVICE_ID);
97     }
98   }
99   PROCESS_END();
100 }
```

Code B.3: Exercise 2. Sender full code.

B.2 Receiver

```
#include "contiki.h"
#include "lib/random.h"
#include "sys/ctimer.h"
#include "sys/etimer.h"
#include "net/uip.h"
#include "net/uip-ds6.h"
#include "net/uip-debug.h"
#include "simple-udp.h"
#include "servreg-hack.h"
#include "net/rpl/rpl.h"
#include <stdio.h>
#include <string.h>

#define DEBUG 0
#define UDP_PORT 1234
#define SERVICE_ID 190
#define SEND_INTERVAL (10 * CLOCK_SECOND)
#define SEND_TIME (random_rand() % (SEND_INTERVAL))

static struct simple_udp_connection unicast_connection;

PROCESS(unicast_receiver_process, "Unicast receiver example process");
AUTOSTART_PROCESSES(&unicast_receiver_process);

static void
receiver(struct simple_udp_connection *c,
         const uip_ipaddr_t *sender_addr,
         uint16_t sender_port,
         const uip_ipaddr_t *receiver_addr,
         uint16_t receiver_port,
         const uint8_t *data,
         uint16_t datalen)
{
    if(DEBUG){

     printf("Message received from: ");
        uip_debug_ipaddr_print(sender_addr);
        printf(" on port %d from port %d with length %d: '%s'\n",
          ↪ receiver_port, sender_port, datalen, data);

    }

printf("\nReceived:'%s'",data);
}
```

```
44 static uip_ipaddr_t *
45 //This is different than with the sender. Now, the set_global_address must
         ↪ return the address, so it is defined to be sent as a direction.
46 // This is becasuse after setting it up, the address is used on the
         ↪ "create_rpl_dag" method (see next).
47
48 set_global_address(void){
49
50   static uip_ipaddr_t ipaddr;
51   int i;
52   uint8_t state;
53
54   uip_ip6addr(&ipaddr, 0xBBBB, 0, 0, 0, 0, 0, 0, 0);
55   uip_ds6_set_addr_iid(&ipaddr, &uip_lladdr);
56   uip_ds6_addr_add(&ipaddr, 0, ADDR_AUTOCONF);
57
58   return &ipaddr;
59 }
60
61 static void create_rpl_dag(uip_ipaddr_t *ipaddr){
62
63     struct uip_ds6_addr *root_if;
64
65     root_if = uip_ds6_addr_lookup(ipaddr);
66     if(root_if != NULL) {
67       rpl_dag_t *dag;
68       uip_ipaddr_t prefix;
69
70       rpl_set_root(RPL_DEFAULT_INSTANCE, ipaddr);
71       dag = rpl_get_any_dag();
72       uip_ip6addr(&prefix, 0xaaaa, 0, 0, 0, 0, 0, 0, 0);
73       rpl_set_prefix(dag, &prefix, 64);
74       PRINTF("created a new RPL dag\n");
75     } else {
76       PRINTF("failed to create a new RPL DAG\n");
77     }
78 }
79
80 PROCESS_THREAD(unicast_receiver_process, ev, data){
81
82   uip_ipaddr_t *ipaddr;
83
84   PROCESS_BEGIN();
85   servreg_hack_init();
86   ipaddr = set_global_address();
87   create_rpl_dag(ipaddr);
88
89   servreg_hack_register(SERVICE_ID, ipaddr);
90   simple_udp_register(&unicast_connection, UDP_PORT, NULL, UDP_PORT, receiver);
91
92   while(1) {
93     PROCESS_WAIT_EVENT();
94   }
95   PROCESS_END();
96 }
```

Code B.4: Exercise 2. Receiver full code.